

Nanoserve

Phase 1 — decisions, trade-offs, and what went wrong

Training a 27M-parameter language model from scratch

This document is the companion to `writups/01-training.md`. That file records *what* was built. This one records **why** — every place a choice was made between two reasonable options, and every place something broke and had to be diagnosed.

The second half is the more valuable half. Anyone can describe a transformer. Far fewer people can say *“I measured 33 tokens/sec, expected 3,500, and here is the four-step diagnosis that got me to 47,000”* — because that only comes from having actually hit the wall.

Result	Value	Notes
Parameters	26,747,392	tied embeddings; untied would be 30.9M
Training tokens	536,174,127	Chinchilla-optimal is 535M — a 0.2% miss
Validation loss	1.1485 nats/token	acceptance bar was < 1.8
Perplexity	3.153	on held-out TinyStories V2
Wall clock	3.27 hours	single RTX 3050 Laptop, 4 GB
Throughput	47,000 tokens/s	2.5× the first working version
Tests	114 passing	including HF-parity and bit-exact resume

Everything below was measured on the machine, not quoted from a paper. Where a number turned out to be wrong, the correction is included rather than the number quietly replaced.

1 · What was built

Three components, each written by hand with no library doing the interesting part.

Component	What it is	Deliberately not used
<code>model/transformer.py</code>	Decoder-only transformer: RMSNorm, RoPE, grouped-query attention, SwiGLU, pre-norm residual blocks	<code>AutoModel</code> , <code>nn.MultiheadAttention</code>
<code>model/tokenizer.py</code>	Byte-level BPE: training loop, merge application, encoder, decoder	<code>tokenizers</code> , <code>sentencepiece</code>
<code>model/train/</code>	Data pipeline and a resumable training loop with gradient accumulation	<code>Trainer</code> , <code>accelerate</code>

The one thing borrowed from a library is `scaled_dot_product_attention`, which is a fused CUDA kernel rather than a model. Writing attention as three matmuls and a softmax would have been a correctness exercise, not a systems one, and it would have thrown away the memory-efficient kernel that Phase 2 needs.

The architecture

d_model	512	vocab	8,192
layers	8	context	1,024
query heads	8	d_ff	1,408 (SwiGLU)
KV heads	2 (GQA)	embeddings	tied

The single most consequential number there is **2 KV heads**. Four query heads share each KV head, so the cache stores 2 heads instead of 8, which puts the KV cache at exactly **4 KB per token**. A 16-token block is 64 KB; a full 1,024-token sequence is 4 MB. Every memory figure in Phase 2's paged allocator is four times better because of a decision made in week one.

2 · Decisions — what was chosen over what

Sixteen places where two options were both defensible. In an interview these are the questions that separate “I followed a tutorial” from “I understood the trade-off”.

1. Tied embeddings

Chose	Sharing one weight matrix between the input embedding and the output head.
-------	--

Over	Untied — giving the output projection its own 4.2M parameters.
------	--

With an 8,192-token vocabulary and `d_model` 512, an untied head costs 4,194,304 parameters — **15% of the entire model** — to learn a mapping the embedding table already encodes. Untied the model is 30.9M; tied it is 26.7M. At this scale that capacity is far better spent in the layers. Tying is also standard for small models for exactly this reason.

Say it like this: “At 27M parameters an untied head is 15% of your budget spent re-learning the vocabulary. Tying it buys eight layers' worth of capacity back.”

2. HuggingFace-compatible parameter names

Chose	Naming weights <code>model.layers.N.self_attn.q_proj.weight</code> , matching HF's Llama exactly.
-------	---

Over	A cleaner naming scheme of my own.
------	------------------------------------

This turns the correctness test from a conversion script into a plain `load_state_dict`. I can instantiate HF's `LlamaForCausalLM` with the same shape, copy my weights straight in, and compare logits. It also means publishing the weights later needs no remapping. The cost is aesthetic; the benefit is a test I would otherwise have skipped because it was fiddly.

Say it like this: “I named the parameters after HF's Llama so my parity test is a plain state-dict load. Making the test cheap is how you guarantee it actually gets written.”

3. A KV-cache protocol, in week one

Chose	Attention calls exactly one method: <code>cache.update(layer_idx, k, v)</code> .
-------	--

Over	Writing attention against a concrete cache and refactoring in Phase 2.
------	--

Phase 2 replaces a contiguous cache with a paged block allocator. If attention knows anything about the cache's internals, that swap becomes a rewrite of the model. Behind a single-method interface it is a new class. This cost about fifteen lines in week one and removes the largest structural risk from week five.

Say it like this: “The paged allocator implements one method. Nothing above it changes — I designed that seam three weeks before I needed it.”

4. Byte-level BPE

Chose	The 256 byte values as the base alphabet.
Over	Character-level or word-level vocabulary with an UNK token.

Byte-level means **every possible input is representable** — there is no UNK token and no out-of-vocabulary code path that can be wrong, because there is no such path at all. An emoji or a Cyrillic character the model has never seen decomposes into bytes it has. This is what GPT-2, GPT-4 and Llama 3 all do.

Say it like this: "Byte-level means UNK doesn't exist. There's no out-of-vocabulary branch to get wrong because there's no branch."

5. Pre-tokenization before BPE

Chose	Splitting text on a regex first; merges never cross those boundaries.
Over	Running BPE directly on the raw byte stream.

Without it, BPE learns ". The" as a single token — the period and the following word glued together — and the vocabulary fills with fragments that only match in one context. A test asserts that every learned token decodes to something the splitter would have produced as a single piece.

Say it like this: "Unconstrained BPE learns punctuation glued to the next word. Pre-tokenization is what stops your vocabulary filling with garbage that only matches in one place."

6. Training BPE on a frequency table

Chose	Collapsing the corpus to unique pre-token → count before merging.
Over	Running the merge loop over the 530M-token stream.

This is the difference between hand-written BPE being practical and being an overnight job. 530,381,032 pre-tokens collapse to **59,888 distinct rows**. Each merge then costs work proportional to the number of *distinct words* containing the pair, not to corpus size. The full 2.07 GB corpus trains in 16 minutes of pure Python.

Say it like this: "You never run BPE over the token stream. Collapse to a frequency table first — 530 million pre-tokens become 60,000 rows and the whole thing runs in Python in minutes."

7. Vocabulary size 8,192

Chose	8,192 tokens.
Over	The 32k–50k that general-purpose models use.

TinyStories deliberately restricts itself to roughly the vocabulary a child would use. Published work on this corpus finds the top 8k tokens cover 475.5M of 476.6M tokens — **99.8%**. A 50k vocabulary would have spent 4.2M extra embedding parameters on tokens that essentially never appear. Measured result: 3.965 bytes/token and **98.1% of words encoding to a single token**.

Say it like this: "8k covers 99.8% of this corpus. A 50k vocabulary would be embedding parameters spent on tokens that never appear."

8. TinyStories V2 over V1

Chose	The GPT-4-only regeneration, 2.08 GB.
-------	---------------------------------------

Over	V1, available as 945 MB of parquet — less than half the download.
------	---

The dataset card states plainly that V1 contains GPT-3.5 generations “of lesser quality” and that V2 is GPT-4 only and larger. Phase 1’s acceptance criterion is “grammatical English stories”, which makes corpus quality the thing most directly under test. Paying 1.1 GB of disk for cleaner supervision was an easy trade.

Say it like this: "V1 mixes in GPT-3.5 output. When your acceptance test is fluency, the data quality is the thing being tested — so I took the bigger, cleaner one."

9. uint16 token storage

Chose	Storing token ids as unsigned 16-bit.
-------	---------------------------------------

Over	int32, the default almost everywhere.
------	---------------------------------------

8,192 ids fit in 16 bits with room to spare, so the token file is **1.0 GB instead of 2.1 GB**. That halves disk, halves page-cache pressure, and halves the bytes read per training step. The risk is silent wraparound if the vocabulary ever exceeds 65,536, so `prepare()` raises rather than truncate.

Say it like this: "Vocab is 8k, so ids fit in uint16 — half the disk and half the read bandwidth per step, with an explicit guard against wraparound."

10. Packed flat token stream

Chose	Concatenating all documents with EOS separators and sampling random windows.
-------	--

Over	Padding each story to a fixed length.
------	---------------------------------------

TinyStories averages ~198 tokens per story against a 1,024-token context. Padding each story would leave roughly **80% of every batch as padding** — 80% of the compute producing no gradient. Packing means every position in every batch carries signal. Windows do sometimes straddle a document boundary, and the EOS token is precisely what teaches the model that the text before it does not predict the text after.

Say it like this: "Stories average 198 tokens against a 1024 context. Padding would have wasted 80% of every batch — so I packed the stream and let EOS mark the boundaries."

11. bf16 with no GradScaler

Chose	bfloat16 mixed precision, no loss scaling.
-------	--

Over	fp16 plus GradScaler, which the project plan assumed.
------	---

The plan was written for a Kaggle T4, which has no bf16 and therefore needs fp16 with dynamic loss scaling — and carries the loss-scale-collapse failure mode that comes with it. The RTX 3050 is Ampere, so bf16 is native, and bf16 carries fp32’s exponent range. **An entire class of failure disappears** along with the code that guards against it.

Say it like this: "The plan assumed a T4, which needs fp16 and a GradScaler. My card is Ampere, so bf16 is native — and the loss-scale-collapse failure mode simply doesn't exist."

12. Weight decay on matrices only

Chose Decaying parameters with 2+ dimensions; norms and biases excluded.

Over Applying weight decay uniformly to everything.

Decaying a LayerNorm gain pulls it toward zero, which does not regularise the layer — it changes the function the layer computes. The same argument applies to biases. Only the projections and the embedding table get decay.

Say it like this: *"Decaying a norm gain isn't regularisation, it's changing what the layer computes. Two parameter groups, decay on matrices only."*

13. Resumability built first, not later

Chose Full checkpoint state from the first commit: weights, optimizer moments, step counter, RNG states, and the data sampler's generator.

Over Saving weights only, and adding the rest when something crashes.

The failure this guards against is not hypothetical: a 3-hour run on a laptop meets thermal events and Windows updates. But the real argument is subtler — a run that reloads weights but not optimizer moments *still trains* and *still shows a falling loss*. It is simply worse than it should have been, and nothing surfaces that except an explicit test.

Say it like this: *"A resume that drops AdamW's moments still trains and the loss still falls. It's just quietly worse. That's why the test asserts bit-identical weights."*

14. Training locally, not on Kaggle

Chose The 4 GB laptop GPU.

Over Kaggle's free T4s, as the project plan specified.

Measured rather than assumed: the plan budgeted ~10 GPU-hours on a T4. The 3050 finished in 3.27 hours. Local also means native bf16, no 12-hour session cap, no 1 GB dataset upload, and a live loss curve. The decision flipped because a measurement contradicted the plan.

Say it like this: *"The plan budgeted ten GPU-hours on a T4. I measured my own card first and it came in at three — so the whole Kaggle pipeline became unnecessary."*

15. micro-batch 8

Chose 8 sequences per forward pass, with gradient accumulation ×4.

Over The largest batch that does not crash — which is 24.

See challenge 4. On Windows, exceeding VRAM does not raise an error; the driver backs the allocation with system RAM over PCIe. Micro-batch 24 "fits" and runs **12× slower**. Picking the largest working size would have selected the single worst option available.

Say it like this: *"The largest batch that runs isn't the fastest. Past VRAM, Windows pages to system RAM instead of failing — 24 fits and runs twelve times slower than 8."*

16. torch.compile

Chose	Fusing the graph with inductor, via triton-windows.
-------	---

Over	Staying in eager mode.
------	------------------------

An eager training step launches roughly 700 tiny kernels. On Windows' WDDM driver each launch costs $\sim 12.9 \mu\text{s}$ against $3\text{--}5 \mu\text{s}$ on bare Linux, so the GPU spends most of its time idle between kernels. Compiling fused them for **1.61×**. Verified numerically equivalent: from the same seed, step 60 gave 5.1541 eager and 5.1538 compiled — fused reduction order, not a behaviour change.

Say it like this: *"Eager launches ~ 700 kernels a step and Windows charges 13 microseconds each. Compiling fused them for 1.6×* — and I checked the loss matched to four decimals before trusting it."

3 · Challenges — what broke, and how it was found

Nine things went wrong. Three of them I caused myself, and one of those I caused twice. The diagnoses are the point: each one is a story with a measurement in it.

1. Decode ran at 1% of the hardware roofline

Symptom	Batch-1 decode measured 33 tokens/sec . The card has $\sim 166 \text{ GB/s}$ of bandwidth and the weights are 51 MB, so the bandwidth roofline is $\sim 3,100 \text{ tokens/sec}$.
---------	--

Cause	Four separate things, found in sequence. (i) The benchmark used 3 warm-up iterations on a GPU sitting at 210 MHz — it never clocked up. (ii) <code>no_grad</code> instead of <code>inference_mode</code> . (iii) A genuine host sync I had written: <code>int(position_ids.max())</code> inside the RoPE table's auto-grow, reading a GPU value on the host once per token. (iv) The real bottleneck — ~ 700 kernel launches per token at $12.9 \mu\text{s}$ each under Windows' WDDM driver.
-------	--

Fix	Proper warm-up, <code>inference_mode</code> , and moving the capacity check to a host-side call. Then <code>torch.compile</code> for the launch overhead itself.
-----	--

Why it is worth telling: The arithmetic closes exactly: $701 \text{ kernels} \times 12.9 \mu\text{s} = 9.04 \text{ ms}$ against 9.4 ms measured — 96% explained. It also contains an honest wrong turn. I first claimed “host - launch == wall proves a sync”. It does not — it is equally consistent with the host being the bottleneck, which is what it actually was. `torch.cuda.set_sync_debug_mode("error")` settled it by raising on any sync, and the decode step passed clean.

2. My optimisation made it 20% slower

Symptom	Replacing a manual <code>repeat_kv</code> with SDPA's native <code>enable_gqa=True</code> — which avoids materialising an 8 MB tensor per token — made the step slower : 9.4 → 11.3 ms, and kernel count rose from 701 to 909.
Cause	The fused attention kernels require query and key to have the <i>same</i> head count. Passing mismatched heads with <code>enable_gqa=True</code> silently drops SDPA to its unfused MATH backend. Materialising the heads is what <i>buys</i> the fused kernel.
Fix	Reverted, with the measurement recorded in a comment so I do not retry it in three weeks.

Why it is worth telling: This is the cleanest possible argument for measuring instead of reasoning. The optimisation was correct on paper — it removed a real copy and 32 real kernel launches — and it was a pessimisation in practice. A backend probe showed why: ['EFFICIENT_ATTENTION', 'MATH'] became ['MATH'].

3. A benchmark that contradicted itself

Symptom	A benchmark measured 28,715 tokens/sec. The real training loop measured 18,800. Re-running the <i>benchmark's own code</i> then gave 435 ms where it had given 285 ms.
Cause	Identical code, different machine state. Thermal sampling during a sustained run showed the GPU was not thermally limited (62 °C) but pinned at exactly 30.0 W with the clock at ~1,000 MHz of a 2,100 MHz maximum. The laptop was on battery, where the GPU enforces 30 W of a 60 W default. On AC it enforces 75 W and clocks at ~1,950 MHz.
Fix	Plug in the charger. Nothing in the code changed.

Why it is worth telling: Both measurements were correct — they measured two different machines. The lesson is that a benchmark result is only meaningful alongside the machine state that produced it, which is exactly why serious benchmark harnesses record hardware and driver versions in their output. The largest single speedup in this project was not a code change.

4. Exceeding VRAM did not fail — it got 12× slower

Symptom	A micro-batch sweep showed sizes 1 through 24 all running, with throughput peaking at 8 and then collapsing : 28,715 → 13,350 → 6,938 → 2,485 tokens/sec.
Cause	On Windows the WDDM driver does not refuse an allocation that exceeds VRAM. It backs it with system RAM over PCIe. Micro-batch 24 reserved 8,586 MB on a 4,096 MB card and ran anyway — 12× slower than micro-batch 8.
Fix	The benchmark now reports the <i>fastest</i> size and flags over-commitment explicitly, rather than reporting the largest that does not raise.

Why it is worth telling: My first version of that script printed “largest fitting micro-batch: 24”, which would have recommended **the single worst option in the table**. A benchmark that measures the wrong quantity is more dangerous than no benchmark, because it carries authority.

5. An RMSNorm bug that returns zeros, not NaN

Symptom	Writing a test to prove the fp32 reduction in RMSNorm was necessary, I asserted the naive fp16 version would produce NaN. The test failed — the output was finite.
Cause	In fp16 an activation of $1e3$ squares to $1e6$, which overflows to inf . The mean is then inf , $\text{rsqrt}(\text{inf})$ is 0 , and the layer returns all zeros .
Fix	The test now asserts the value, not just finiteness — because finiteness passes.

Why it is worth telling: This is worse than a NaN in every way that matters. A NaN propagates loudly and stops the run. Silent zeros annihilate the activation, nothing raises, no NaN appears, and the only symptom is a model that does not learn as well as it should. I found it by writing a test to prove my own code was necessary.

6. The dataset's own train/valid split leaks

Symptom	Checking the downloaded files, the train file ends ...She said, "Yo and the validation file begins u don't have to be scared of the loud dog...
Cause	The official split is a byte offset, not a document boundary . It cuts a story in half — the same story appears on both sides.
Fix	<code>drop_last_document</code> on train and <code>drop_first_document</code> on validation.

Why it is worth telling: One story in 2.7 million is a negligible leak, and the honest answer is that it would not have changed any number in this report. But it cost four lines to remove, and “I checked the boundaries of my train/test split” is a different sentence from “I assumed the published split was clean”.

7. best.pt pointed at a worse model

Symptom	After training, loading <code>best.pt</code> reported step 16,000 — not the final step 16,364.
Cause	The final evaluation in the training loop ran <i>after</i> the last checkpoint save and was never compared against <code>best_val</code> . A run that improves on its closing steps therefore leaves <code>best.pt</code> pointing at a strictly worse model.
Fix	Compare the final eval too. Measured over 200 validation batches: step 16,000 gave 1.1496, step 16,364 gave 1.1485.

Why it is worth telling: The gap is small — 0.001 nats. The failure mode is not: this is precisely the bug that publishes the wrong weights to a public model hub, and nobody would ever notice.

8. Task Manager reported 0% GPU during training

Symptom	With training running and <code>nvidia-smi</code> reporting 100% utilisation, Windows Task Manager showed the GPU at 0%.
Cause	Task Manager's default graphs read the 3D engine and exclude CUDA. Confirmed on the machine rather than taken on faith: Windows' own GPU Engine performance counter read <code>3d = 19.3%</code> at the same instant <code>nvidia-smi</code> read <code>util = 100%</code> .
Fix	Nothing to fix in the code. In Task Manager, right-click the GPU graph and switch a pane to Cuda .

Why it is worth telling: Worth knowing because it is a false alarm that looks exactly like a real one, and acting on it — “the GPU is idle, something is broken” — would send you debugging a problem that does not exist.

9. torch.compile poisons your checkpoint keys

Symptom	<code>torch.compile</code> returns a wrapper module. Its <code>state_dict</code> keys are prefixed with <code>_orig_mod.</code>
Cause	Saving the compiled wrapper would make every checkpoint incompatible with an uncompiled run — and with the export used to publish the weights.
Fix	Keep the uncompiled module as the thing that is saved and loaded; compile a separate reference and use that only for the forward pass. Asserted in a test: 75 weight keys, 0 polluted, and the checkpoint loads into a plain model.

Why it is worth telling: A one-line trap with a delayed blast radius. You would not discover it until weeks later, when the weights refuse to load somewhere else.

4 · How to talk about this

Four questions an interviewer is likely to ask, and the honest answer to each.

“Walk me through the project.”

I trained a 27-million-parameter language model from scratch — the transformer, the tokenizer and the training loop are all hand-written — and I am now building the inference engine and distributed serving layer around it. Phase 1 finished at 1.15 nats/token on held-out data in 3.3 hours on a 4 GB laptop GPU. The model being small is deliberate: it means the distributed system in Phase 3 is actually testable on hardware I own.

“How did you know your implementation was correct?”

Loss going down is far too weak a signal — a model with a wrong RoPE or an off-by-one causal mask still trains, just worse, and you find out three days into a run. So I built the same architecture in HuggingFace's Llama, copied one set of weights into both, and required the logits to agree to $2e-5$ and greedy generation to match token for token. Then I tested the failure that still produces fluent text: decoding position 412 with 411 cached, and 256 sequential decode steps checked at every position rather than only the last.

“Tell me about something that went wrong.”

The best one is the 33 tokens/sec story — challenge 1 — because it has four layers and I got the first diagnosis wrong. The short version: I claimed a symptom proved there was a host sync; it did not, it was equally consistent with the host simply being the bottleneck. I stopped inferring and used `set_sync_debug_mode("error")`, which raises on any sync. It passed clean. The real answer was 700 kernel launches per token at 13 microseconds each — and the arithmetic closed to 96%.

The second-best is challenge 3, because the largest speedup in the project was plugging in a charger. A benchmark and the real loop disagreed by 50%; the code was identical; the GPU was power-capped at 30 W on battery versus 75 W on AC. It is a good story precisely because it is not a clever one — it is a reminder that a benchmark number without its machine state is not a number.

“What would you do differently?”

Two things. I wrote the same class of bug twice — a host synchronisation in a hot loop — and the second time was *after* I had already found and fixed the first. That should have become a checklist item the moment I found it, not a lesson I had to learn again.

And I would benchmark on Linux from the start. Batch-1 decode on this machine is launch-bound, and Windows' WDDM driver charges 3–4× what bare Linux does per kernel launch. That inflates the apparent benefit of batching by roughly 2–3×. It does not affect any Phase 1 number, but it would have quietly corrupted every Phase 2 comparison, so the serving benchmarks are going on Linux.

Numbers worth having memorised

Number	What it is
26,747,392	parameters — tied embeddings; 30.9M untied
4 KB / token	KV cache, thanks to 2 KV heads. 16-token block = 64 KB
536M / 535M	training tokens vs Chinchilla-optimal — a 0.2% miss
1.1485	final validation loss, against an acceptance bar of 1.8
3.965 bytes/token	tokenizer compression; 98.1% of words are one token

701 × 12.9 μs	kernels per step × WDDM launch cost = the eager bottleneck
30 W → 75 W	battery vs AC power cap — the largest single speedup
2.5×	total throughput gain: 18,800 → 47,000 tokens/sec

5 · What Phase 2 inherits

Four things were built in Phase 1 specifically to make Phase 2 tractable:

Asset	Why it matters in Phase 2
4 KB/token KV cache	The number the paged block manager is sized against. GQA made it 4× smaller than it would otherwise be.
KVCache protocol	The paged allocator implements one method. The model does not change.
model/generate.py	Deliberately plain. The scheduler, paged cache and continuous batching must reproduce it token for token under greedy decoding.
Block-boundary tests	Decode already asserted at positions 15, 16, 17, 63, 64, 65 — exactly the 16-token edges the allocator will cross.

And one warning carries forward: the serving benchmarks belong on Linux, because batch-1 decode on this machine is launch-bound and would overstate the benefit of continuous batching by 2–3×. The whole point of those numbers is that they survive scrutiny.

Generated from scripts/make_report.py. All figures measured on an RTX 3050 Laptop (4 GB, Ampere), Windows 11, PyTorch 2.11.0+cu128.